StudAura

Watch. Capture. Track. Revise.

A YouTube-based learning workspace, shipped with a complete DevSecOps pipeline

Note: the app walkthrough comes first - skip to slide 13 for the DevOps part.

Docker · GitHub Actions · Trivy · GHCR · Terraform · EKS · Helm · ArgoCD · Prometheus · Grafana

Avishkar Chavan · DevOps final capstone

What StudAura does

Learning paths

Import videos or whole playlists; failures are reported per item.

Player

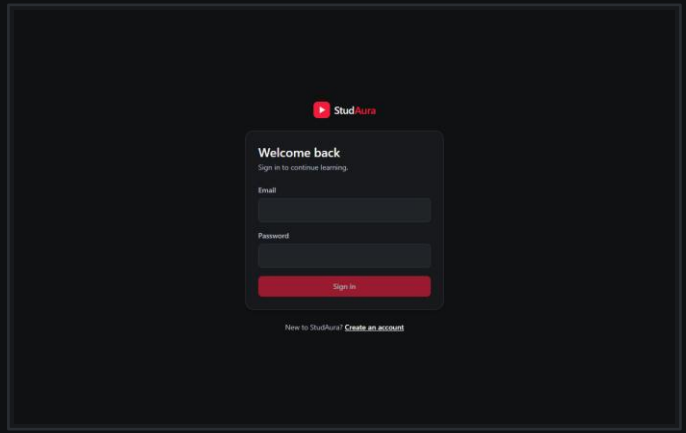
Embedded YouTube player that resumes where you stopped.

Notes

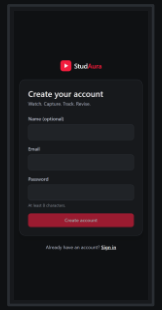
Rich text with autosave, retry and conflict detection.

Goals + revision

Daily goal from real playback; a revision queue.

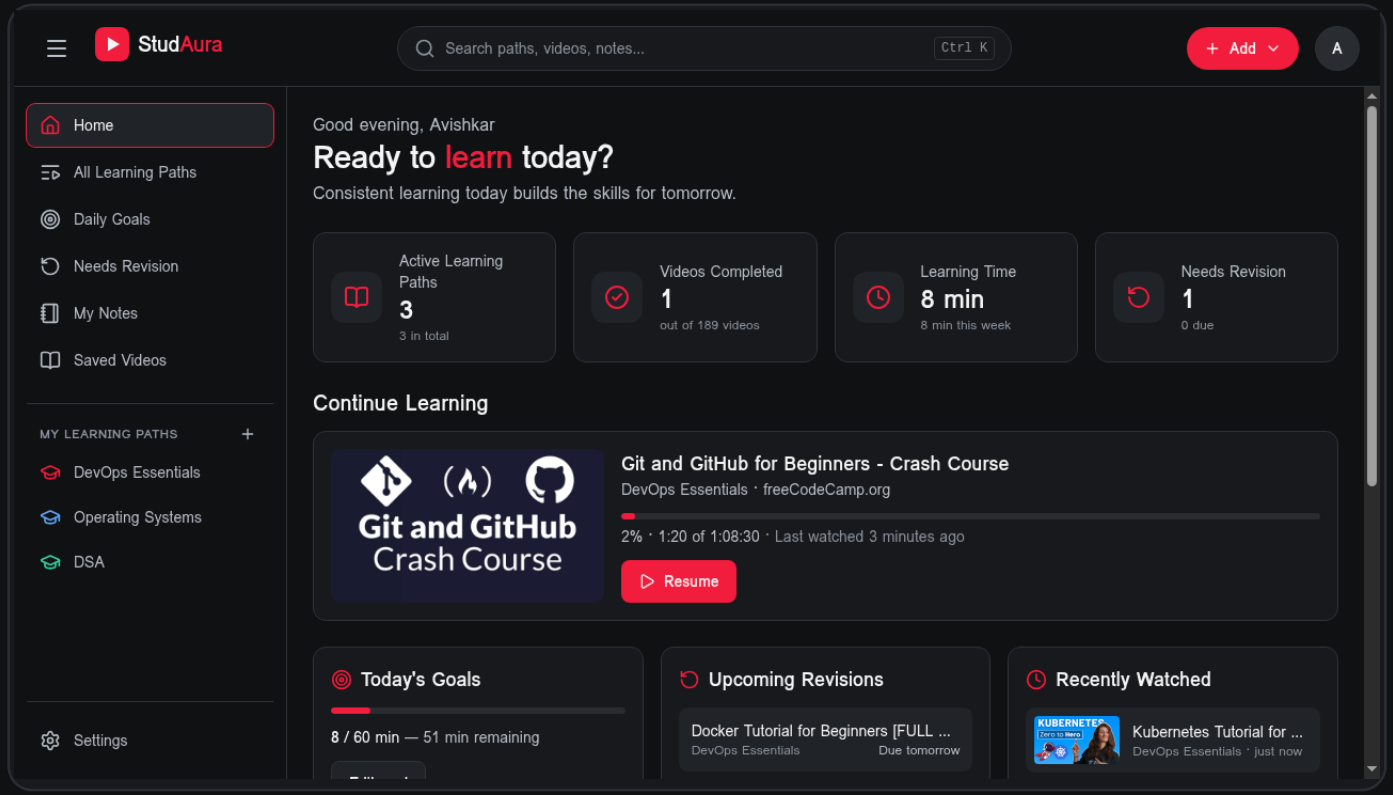


Docker Compose, localhost:3000



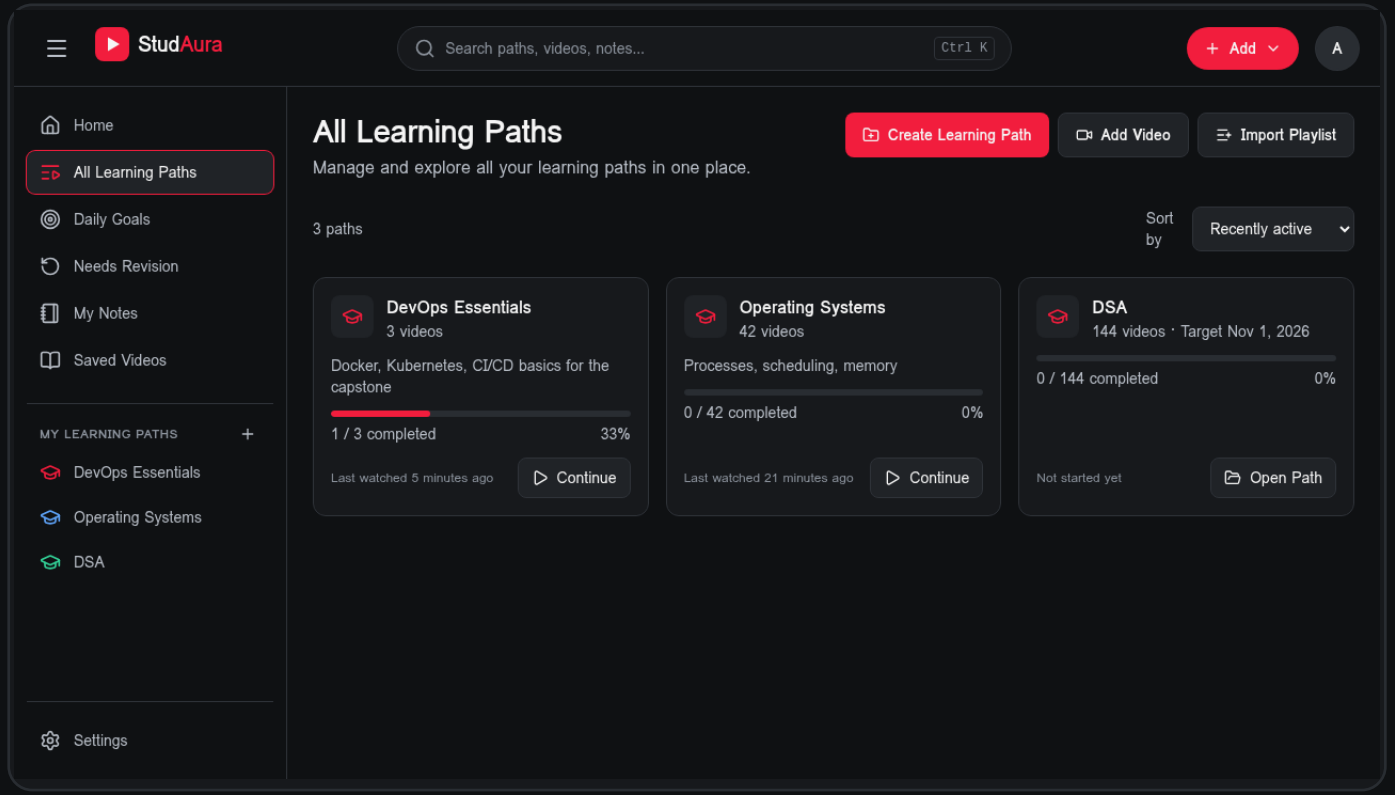
The same app through the EKS load balancer, on a phone-sized screen

Home dashboard



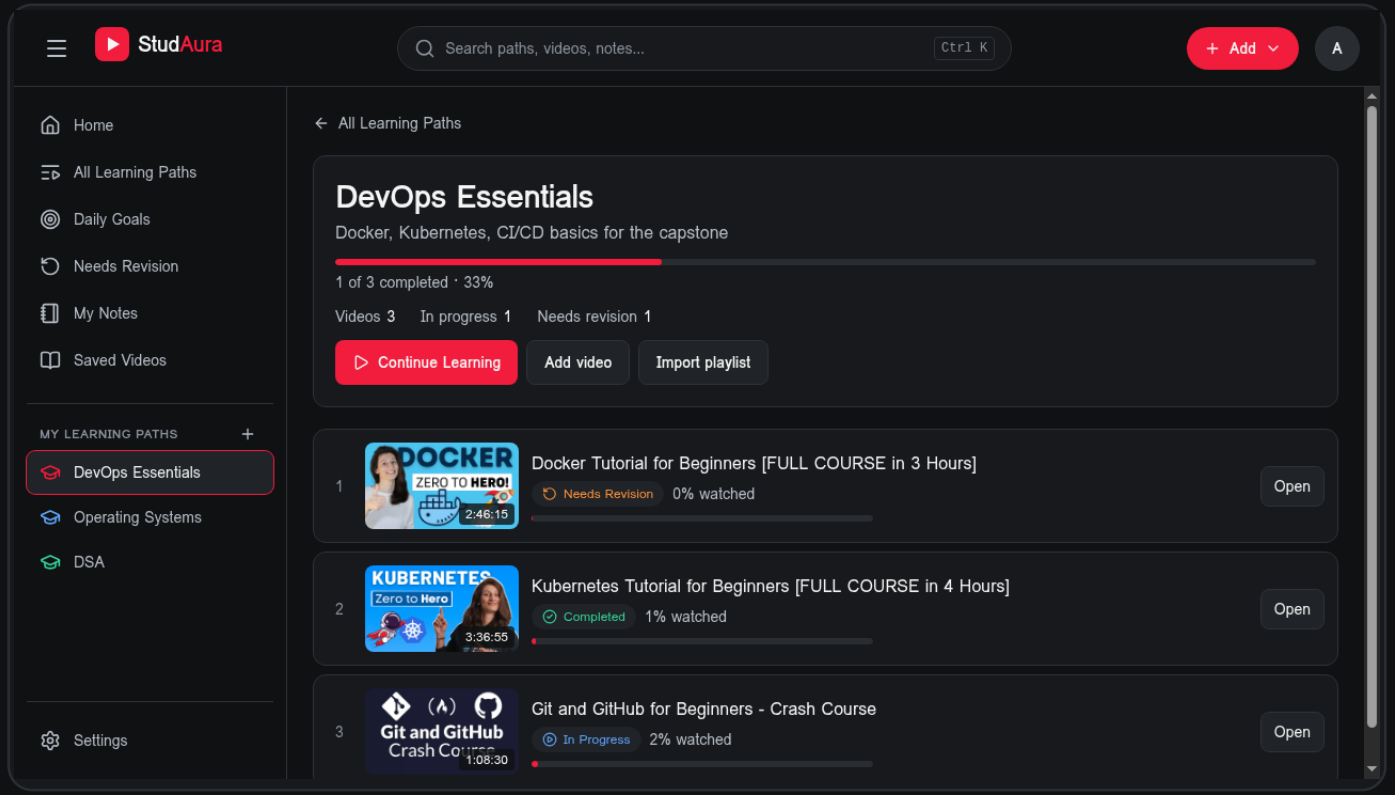
Progress at a glance: stats, today's goal, one-click resume

Learning paths



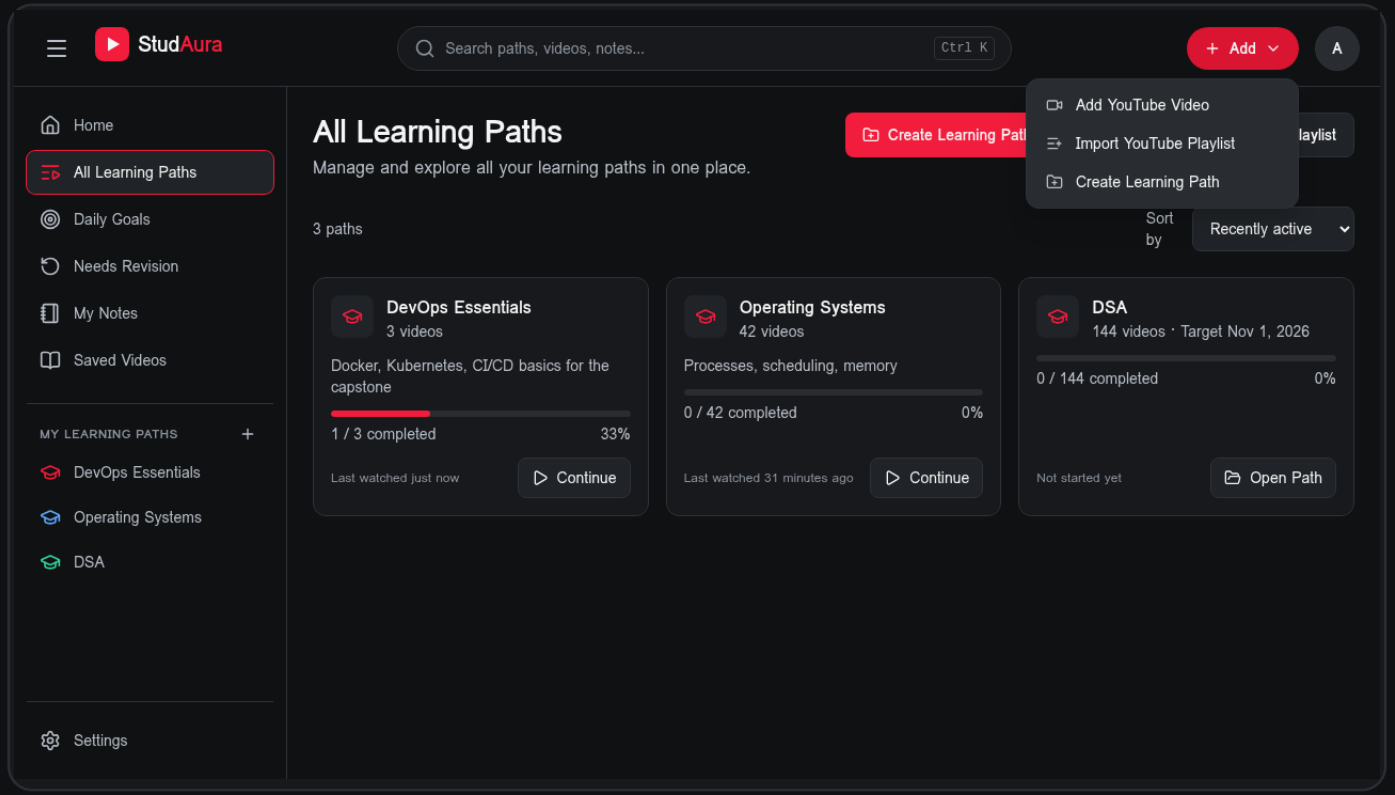
Videos and playlists grouped into paths, with completion tracked

Inside a learning path



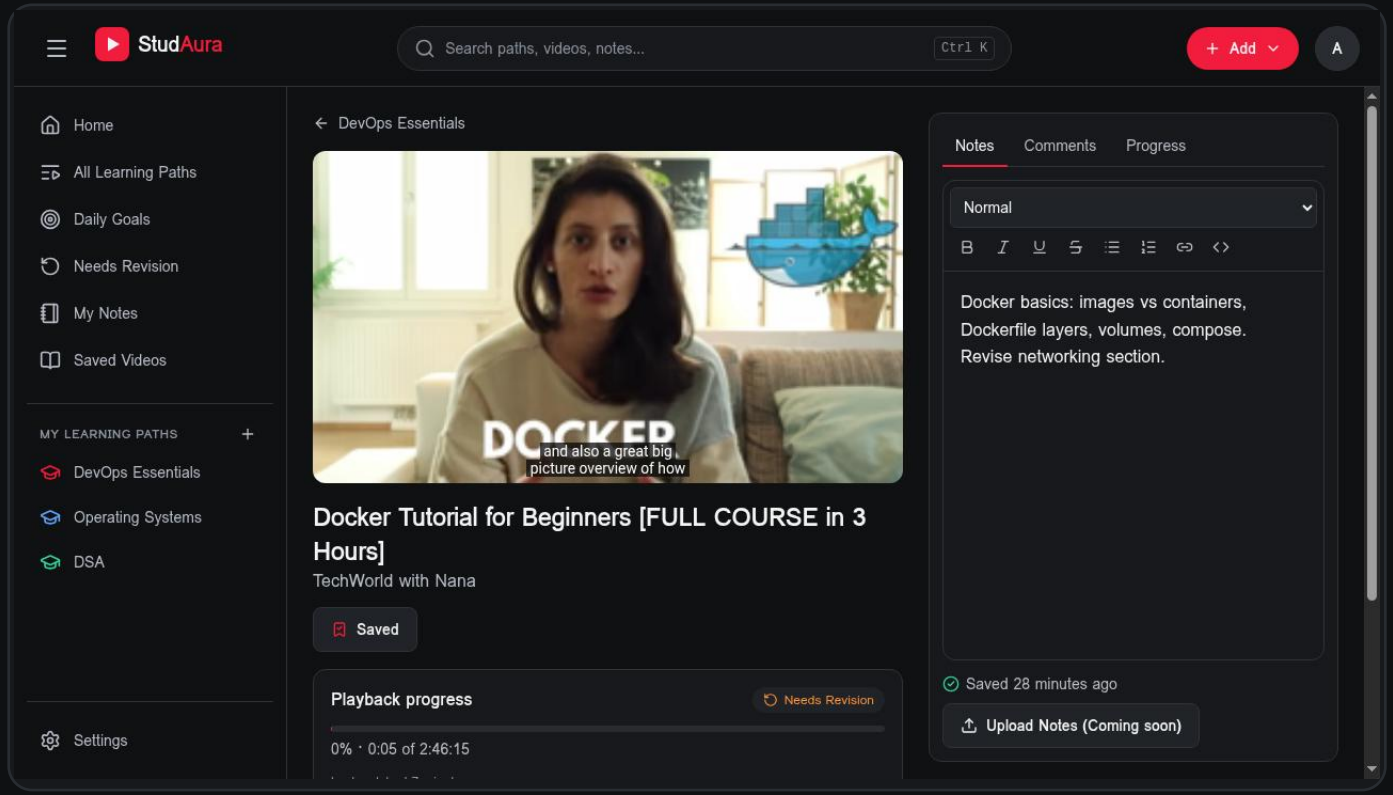
Each video shows its status and watch progress

Adding content



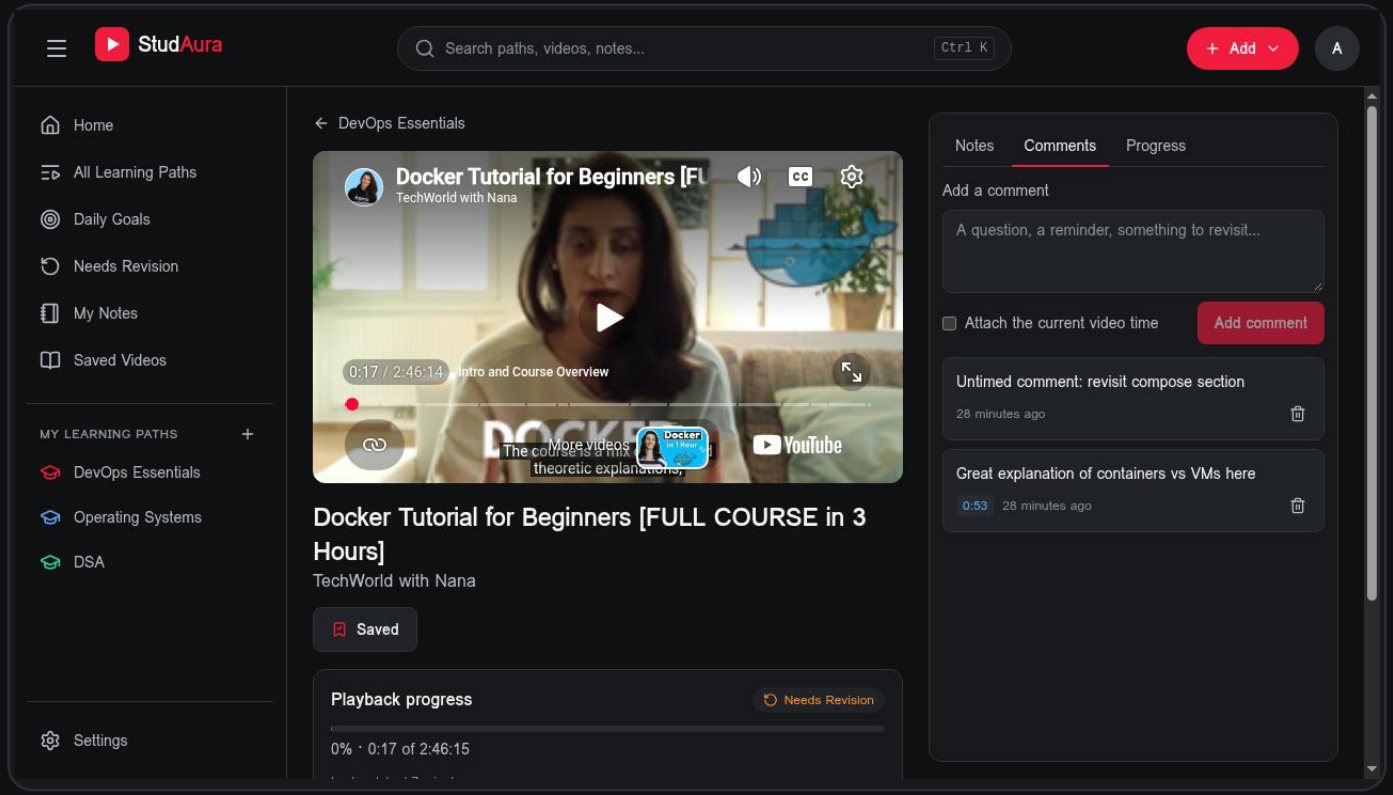
Import one YouTube video or a whole playlist

Watch and take notes



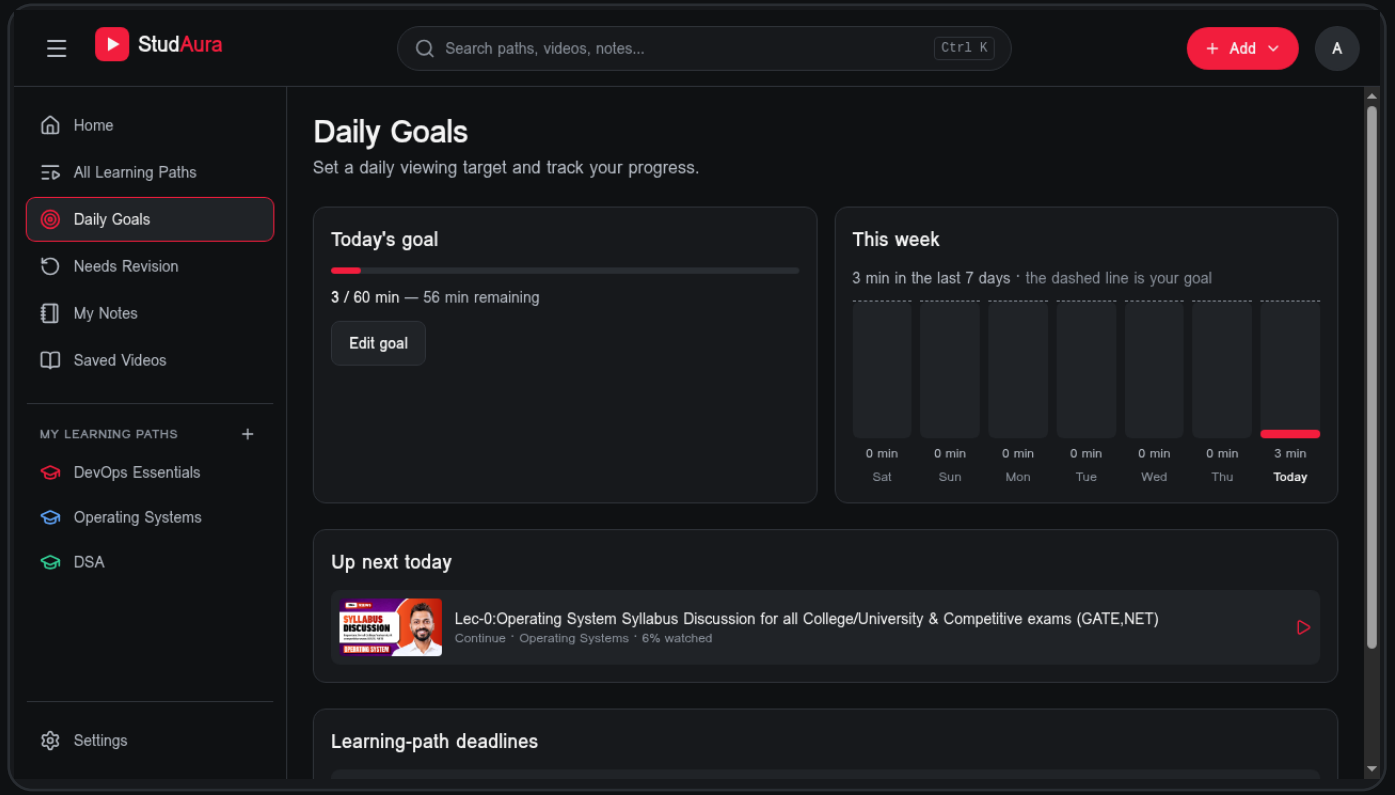
The player and notes editor sit side by side; notes autosave

Timestamped comments



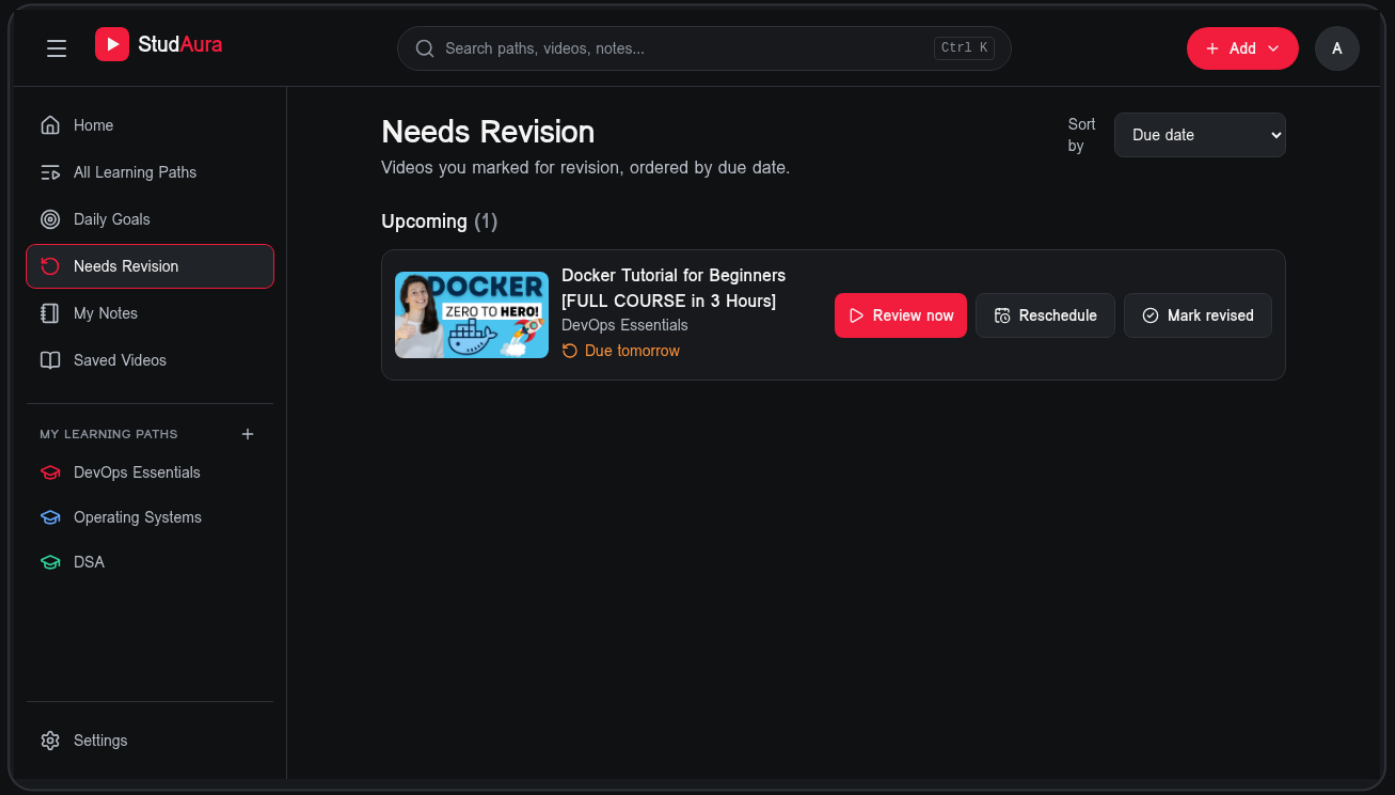
Comments can point to an exact moment in the video

Daily goals



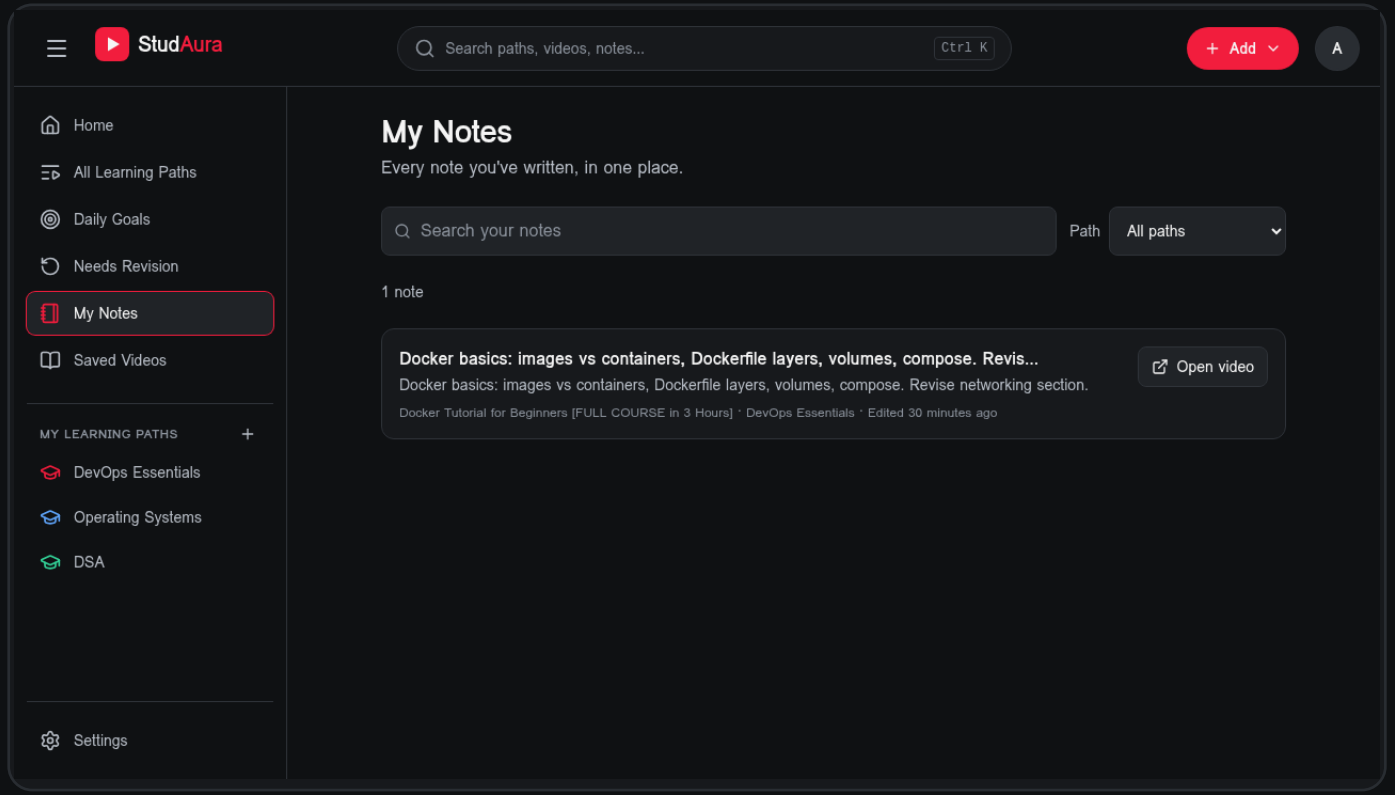
Goals count real playback time, with the week shown as a chart

Revision queue



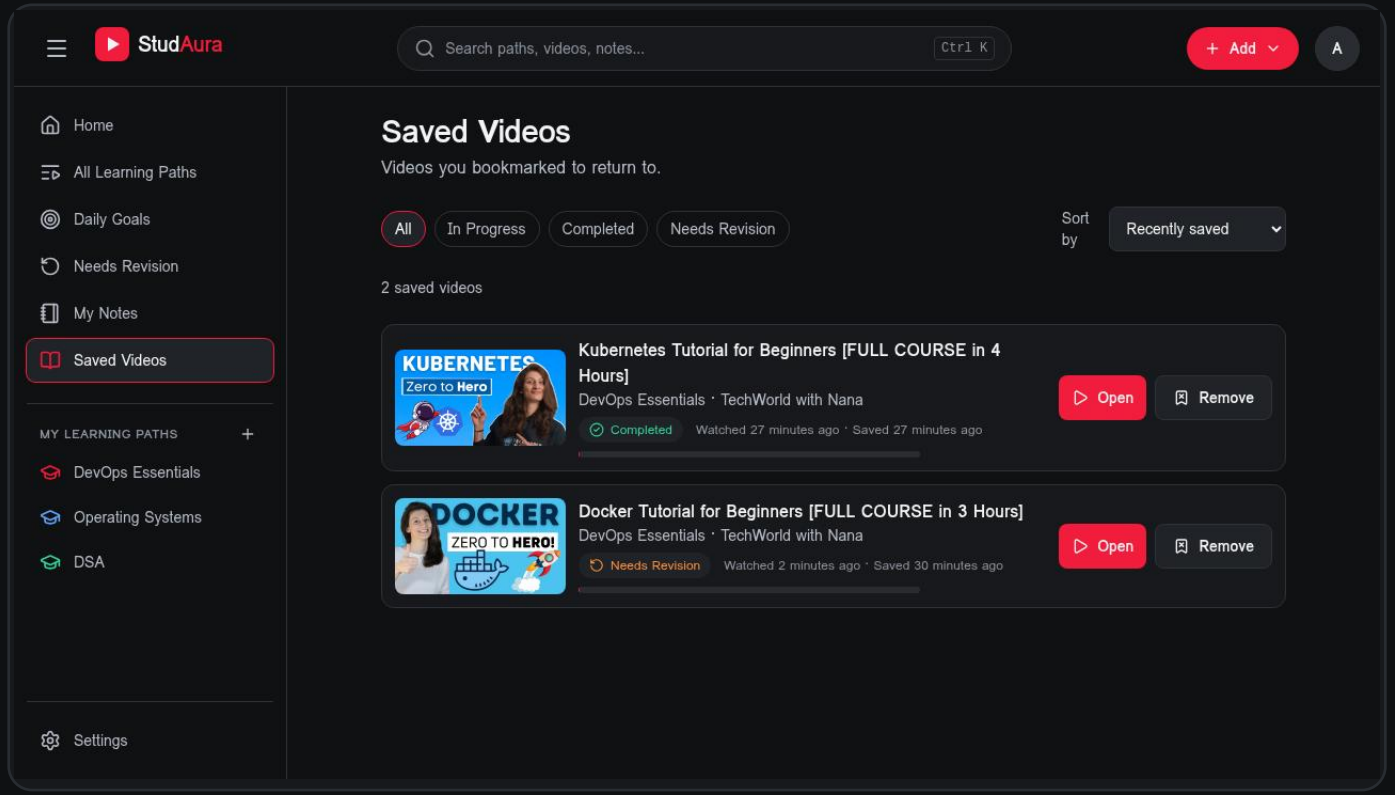
Flag a video for revision and review it by its due date

My notes



Every note in one searchable library

Saved videos



Bookmarked videos with status filters

Tech stack

Frontend

React 18
Vite + TypeScript
Tailwind CSS v4
TanStack Query
TipTap editor
Vitest + Playwright

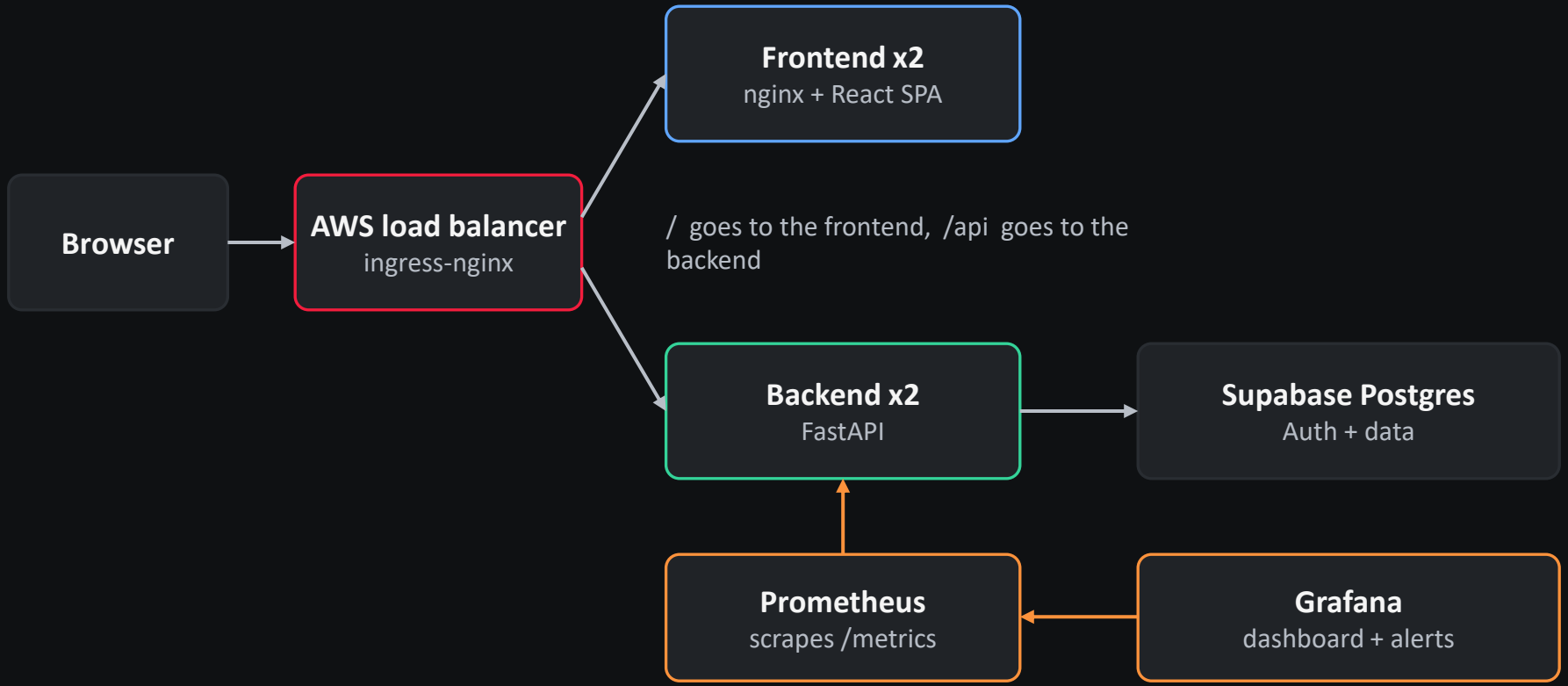
Backend

FastAPI
SQLAlchemy 2 (async)
Alembic migrations
JWT via Supabase JWKS
PostgreSQL
prometheus-fastapi-instrumentator

Delivery

Docker (multi-stage)
GitHub Actions
Trivy, GHCR
Terraform, AWS EKS
Helm, ArgoCD
Prometheus, Grafana

Architecture



CI/CD pipeline

Pull request
lint, tests, build

→

Images
build, non-root

→

Publish
Trivy, then GHCR

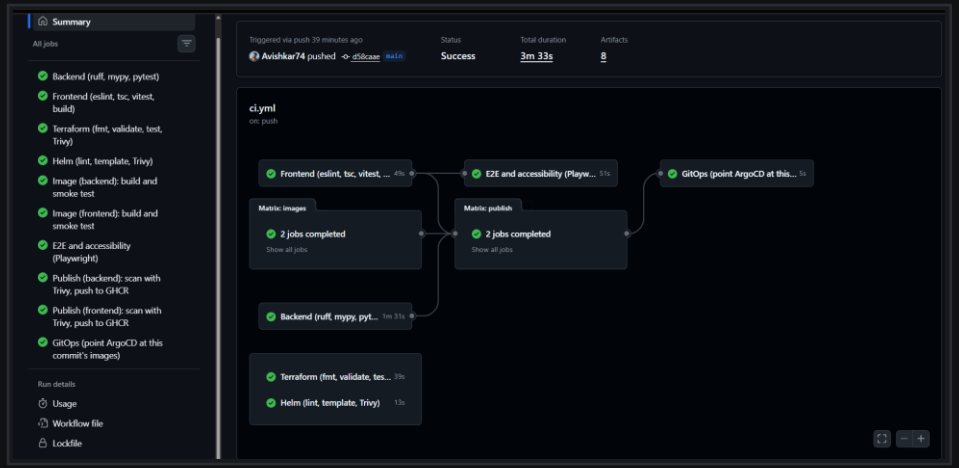
→

GitOps
deploy branch

→

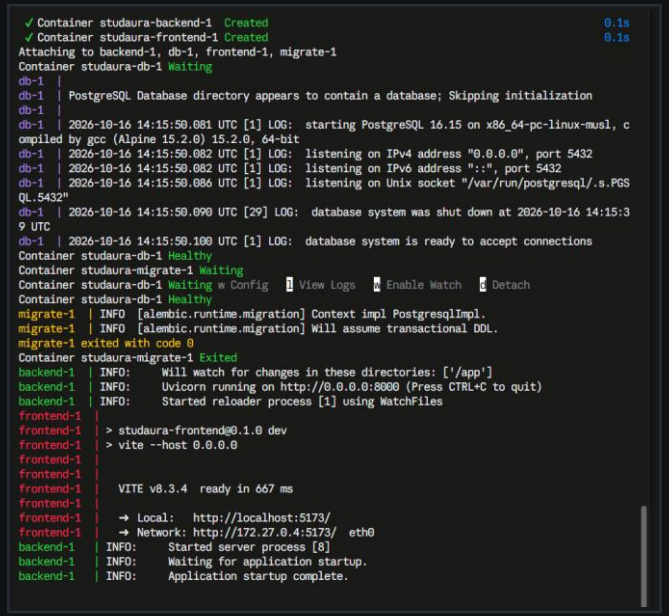
ArgoCD
syncs cluster

- Runs on every pull request and every push to main
- pytest on a real PostgreSQL service; a skipped test fails the build
- Images tagged with the commit SHA only, never latest
- Playwright + axe accessibility checks on main

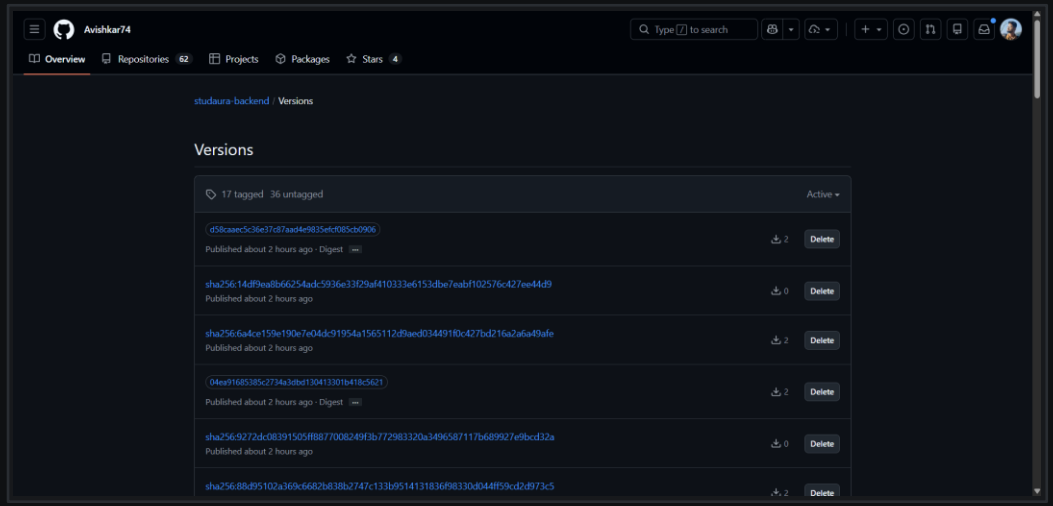


Green run on main: all jobs passed

Containers and registry



docker compose up --build



GHCR: studaura-backend, commit-SHA tags

- Multi-stage Dockerfiles, both images run as non-root
- Tags are commit SHAs; no latest tag is published

DevSecOps with Trivy

- Both images are scanned before they are pushed
- A fixable HIGH or CRITICAL blocks the publish
- Terraform and rendered Helm manifests are scanned on every PR
- Non-root, read-only filesystem, dropped capabilities
- No secrets in Git; actions pinned to commit SHAs

C: > Users > chava > AppData > Local > Temp > 0400e50b-81a7-402a-bf87-91d26e2cc3ed_trivy-backend.zip.trivy-backend.zip > trivy-backend.txt

Report Summary

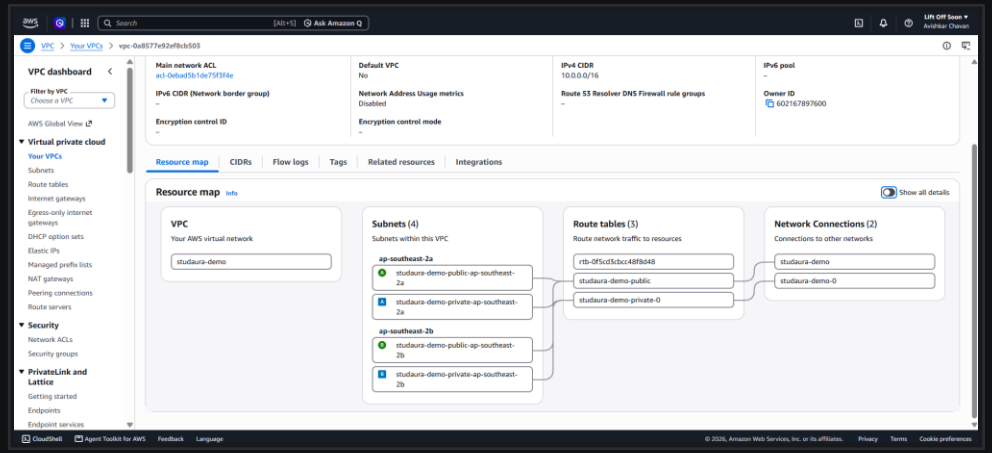
Target	Type	Vulnerabilities	Secrets
ghcr.io/avishkar74/studaura-backend:d58caaec5c36e37c87aad4e9835efcf085cb0906 (debian 13.7)	debian	0	-
opt/venv/lib/python3.12/site-packages/alembic-1.20.0.dist-info/METADATA	python-pkg	0	-
opt/venv/lib/python3.12/site-packages/annotated_doc-0.0.5.dist-info/METADATA	python-pkg	0	-
opt/venv/lib/python3.12/site-packages/annotated_types-0.8.0.dist-info/METADATA	python-pkg	0	-
opt/venv/lib/python3.12/site-packages/anyio-4.15.1.dist-info/METADATA	python-pkg	0	-
opt/venv/lib/python3.12/site-packages/asyncpg-0.32.0.dist-info/METADATA	python-pkg	0	-
opt/venv/lib/python3.12/site-packages/certifi-2026.7.22.dist-info/METADATA	python-pkg	0	-
opt/venv/lib/python3.12/site-packages/cffi-2.1.1.dist-info/METADATA	python-pkg	0	-
opt/venv/lib/python3.12/site-packages/click-8.5.0.dist-info/METADATA	python-pkg	0	-
opt/venv/lib/python3.12/site-packages/cryptography-50.0.2.dist-info/METADATA	python-pkg	0	-
opt/venv/lib/python3.12/site-packages/ecdsa-0.19.2.dist-info/METADATA	python-pkg	0	-
opt/venv/lib/python3.12/site-packages/fastapi-0.143.2.dist-info/METADATA	python-pkg	0	-

Backend image: 0 vulnerabilities

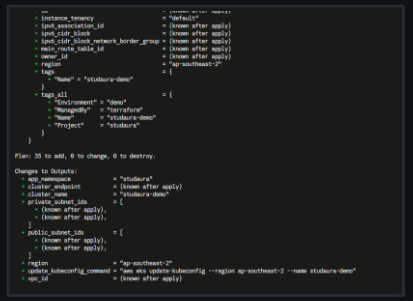
The first run of the HIGH gate caught two Starlette CVEs; upgrading cleared them.

Terraform: VPC and EKS as code

- VPC with 2 public and 2 private subnets in 2 zones
- Internet gateway and a NAT gateway for the nodes
- EKS cluster and a managed node group
- IAM roles, KMS key, add-ons
- terraform.tfvars.example, no credentials in Git
- Plan tests run in CI



AWS console: VPC resource map, Sydney



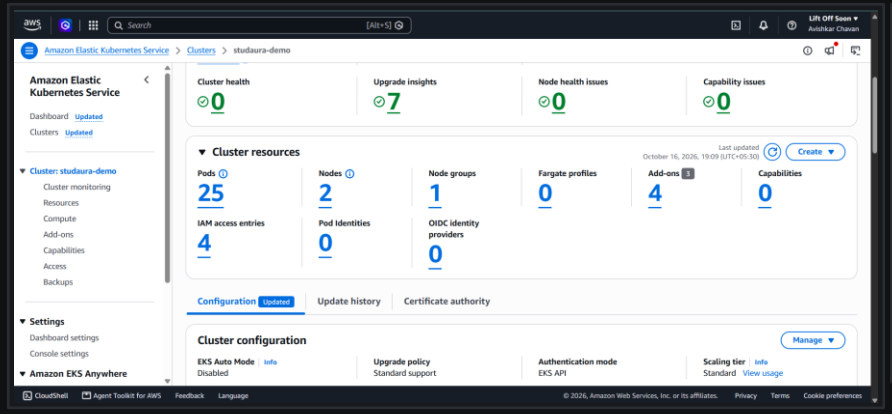
```
Plan: 35 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + app_namespace       = "studaura"
  + cluster_endpoint    = (known after apply)
  + cluster_name        = "studaura-demo"
  + private_subnet_ids  = [
      + (known after apply),
      + (known after apply),
    ]
  + public_subnet_ids   = [
      + (known after apply),
      + (known after apply),
    ]
  + region              = "ap-southeast-2"
  + update_kubeconfig_command = "aws eks update-kubeconfig --region ap-southeast-2 --name studaura-demo"
  + vpc_id              = (known after apply)
```

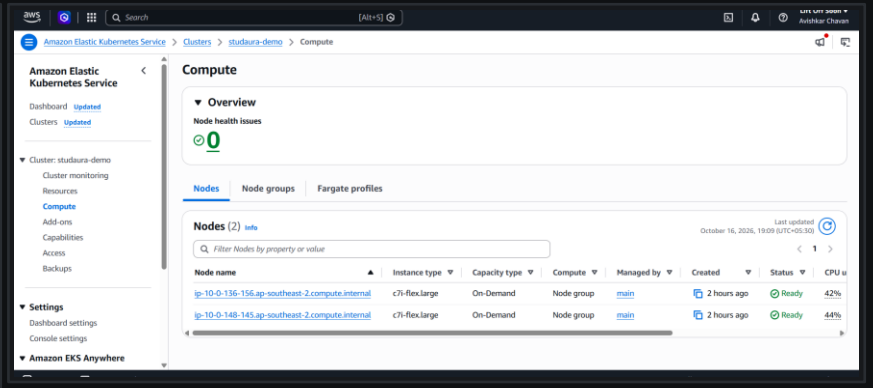
35

resources in one plan

Terraform: applied and destroyed



EKS cluster studaura-demo, Active



Node group: 2 x c7i-flex.large, Ready

```
erPolicy]
aws_cloudwatch_log_group.cluster: Destroying... [id=/aws/eks/studaura-demo/clus
aws_subnet.private[1]: Destroying... [id=subnet-0b42350449937ff37]
aws_subnet.private[0]: Destroying... [id=subnet-0cefde42983e4b7e3]
aws_kms_key.eks_secrets: Destroying... [id=4a1c2b8c-5bcc-4779-a229-cc2052f43643
aws_iam_role_policy_attachment.cluster: Destruction complete after 3s
aws_iam_role.cluster: Destroying... [id=studaura-demo-eks-cluster]
aws_kms_key.eks_secrets: Destruction complete after 4s
aws_cloudwatch_log_group.cluster: Destruction complete after 4s
aws_iam_role.cluster: Destruction complete after 1s
aws_subnet.private[0]: Destruction complete after 5s
aws_subnet.private[1]: Destruction complete after 5s
aws_vpc.main: Destroying... [id=vpc-0a8577e92ef8cb503]
aws_vpc.main: Destruction complete after 2s

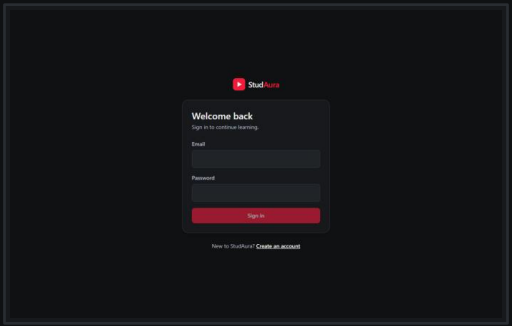
Destroy complete! Resources: 35 destroyed.
PS D:\Resume projects\StudAura\deploy\terraform>
```

terraform destroy: 35 destroyed

- init, plan and apply all run from the CLI
- Destroyed once the evidence was captured, so nothing keeps billing

Kubernetes and Helm

- 2 backend + 2 frontend pods, autoscaling 2 to 4
- ClusterIP Services; one Ingress
- / goes to the frontend, /api to the backend
- Migration runs as a hook Job



The app through the Ingress

```
PS D:\Resume projects\StudAura> kubectl get pods,svc,ingress -n studaura
NAME                                    READY   STATUS    RESTARTS   AGE
pod/studaura-backend-865ccd99d8-gbszt   1/1     Running   0          64m
pod/studaura-backend-865ccd99d8-t7frk   1/1     Running   0          64m
pod/studaura-frontend-69dcbb88fd-5tx6z  1/1     Running   0          64m
pod/studaura-frontend-69dcbb88fd-rkwvx  1/1     Running   0          64m

NAME                         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
service/studaura-backend     ClusterIP   172.20.245.61   <none>        80/TCP    129m
service/studaura-frontend    ClusterIP   172.20.186.18   <none>        80/TCP    129m

NAME                                  CLASS   HOSTS   ADDRESS
                                        PORTS   AGE
ingress.networking.k8s.io/studaura    nginx   *       a4f65d523ff5b465faab6fd5d57fe662-93b4ff512e9e
b19c.elb.ap-southeast-2.amazonaws.com   80      129m
PS D:\Resume projects\StudAura>
```

kubectl get pods,svc,ingress -n studaura

```
PS D:\Resume projects\StudAura> helm list -A
NAME            NAMESPACE       REVISION        UPDATED                                 STATUS   CHART                           APP VERSION
argocd          argocd          1               2026-10-16 17:37:22.3547612 +0530 IST   deployed argo-cd-10.10.5                 v3.5.5
ingress-nginx   ingress-nginx   1               2026-10-16 17:31:34.0970408 +0530 IST   deployed ingress-nginx-4.15.1            1.15.1
monitoring      monitoring      2               2026-10-16 18:30:05.5992033 +0530 IST   deployed kube-prometheus-stack-92.9.0    v0.94.1
PS D:\Resume projects\StudAura>
```

helm list -A

GitOps with ArgoCD

main
merge

→

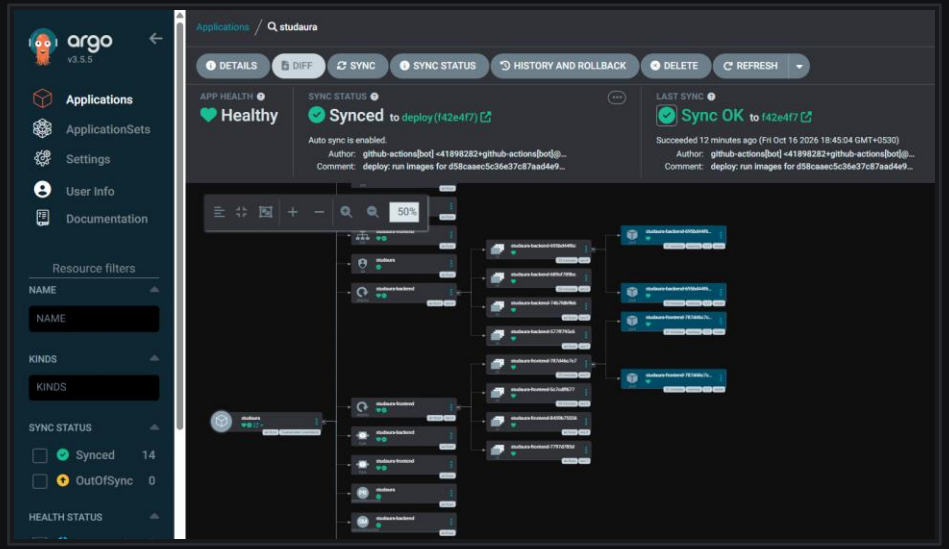
CI publish
images in GHCR

→

deploy branch
image.yaml: SHA

→

ArgoCD
sync + self-heal



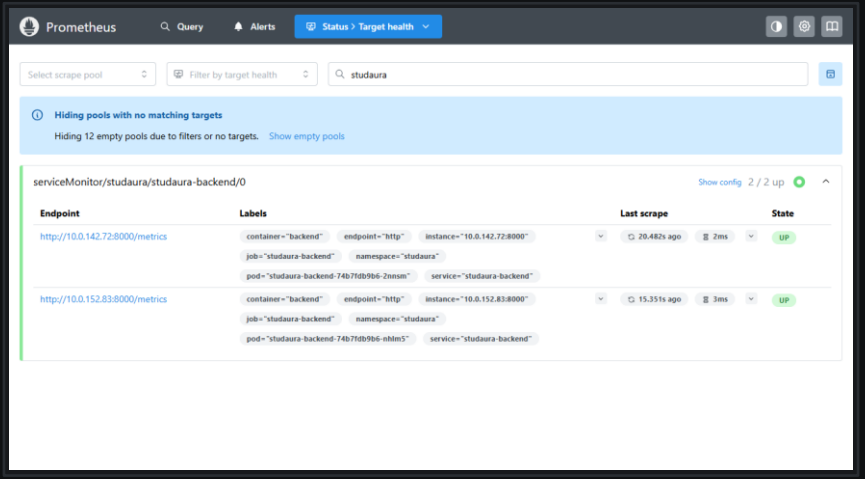
ArgoCD: Healthy, Synced to the deploy branch

- The cluster pulls its state from Git
- CI holds no cluster credentials
- Self-heal reverts manual drift

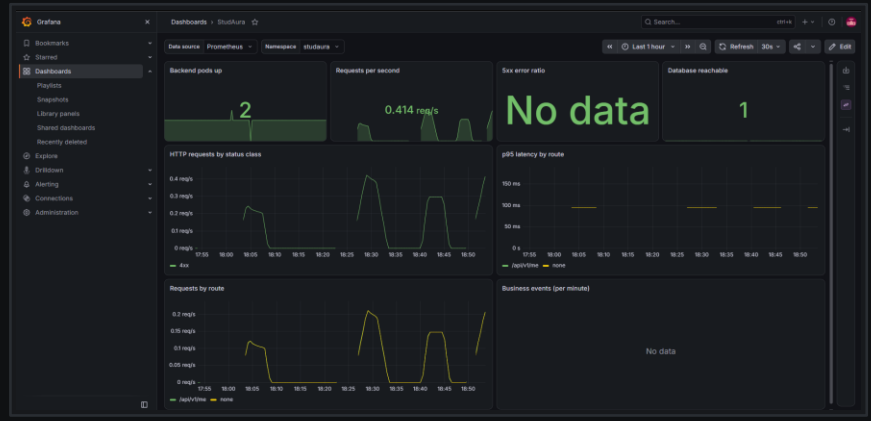
```
PS D:\Resume projects\StudAura> kubectl -n argocd get application studaura
NAME       SYNC STATUS   HEALTH STATUS
studaura   Synced        Healthy
PS D:\Resume projects\StudAura>
```

kubectl get application

Observability

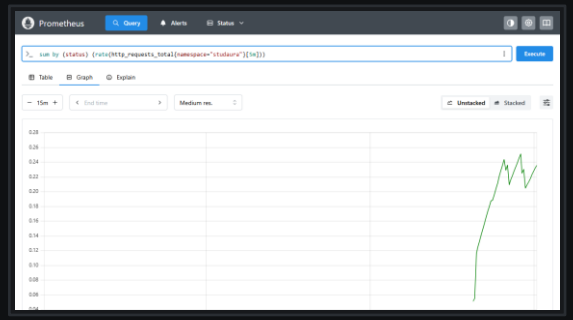


Prometheus: both backend pods UP



Grafana dashboard StudAura

- ServiceMonitor scrapes /metrics
- Alerts: backend down, 5xx, database
- Dashboard provisioned from Git



Live request rate by status class

Quality gates

674

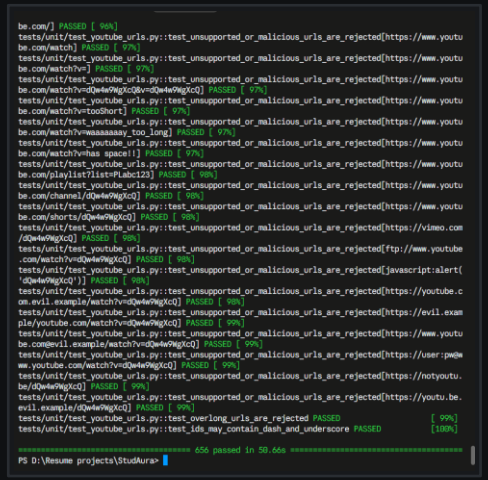
backend tests, real PostgreSQL

425

frontend unit tests

37

pull requests merged



- pytest -v: 674 passed (656 in the captured run)
- Backend tests use a throwaway database and a fake YouTube adapter
- Playwright runs on desktop and phone sizes, with axe WCAG 2 A/AA scans
- Both images scan clean: zero fixable HIGH or CRITICAL findings

Key learnings and limits

What I learned

Small, reviewed pull requests keep CI green.
A pipeline is only trustworthy if every gate can fail.
GitOps removes cluster credentials from CI.
Cloud account policies shape the architecture as much as the code.

Known limits

HTTP only: no domain or certificate yet.
Database in Mumbai, cluster in Sydney: about 260 ms per query.
Prometheus data is not persisted.
The cluster is destroyed once the evidence is captured, to avoid cost.